

FlagTree 如何接入一种 AI 芯片后端

2025 年 8 月

多元 AI 芯片的算子编译器统一生态开源共建

1. **汇聚生态**：防止生态发散从编译器向算子层蔓延，以此为出发点
2. **后端接入**：持续新增后端接入，类型涵盖各类芯片
3. **仓库-中间层-编程接口统一**：以多元 AI 芯片统一编译器仓库为基础，对多后端从编程接口扩展到中间层转换进行横向对齐设计，至少做到编程接口形式统一、中间层以上编译路径分类统一
4. **生态-性能-灵活性**：既要保持和发展 Triton 生态，又要支撑特定硬件属性关联的编译指导和优化，并提升后端友好的算子编写的灵活性，从这三个方面在易用性和性能之间寻求平衡
5. **生态目标**：最终目的是在生态、易用、性能上对各 AI 芯片提供算子库及编译器的软件栈支撑

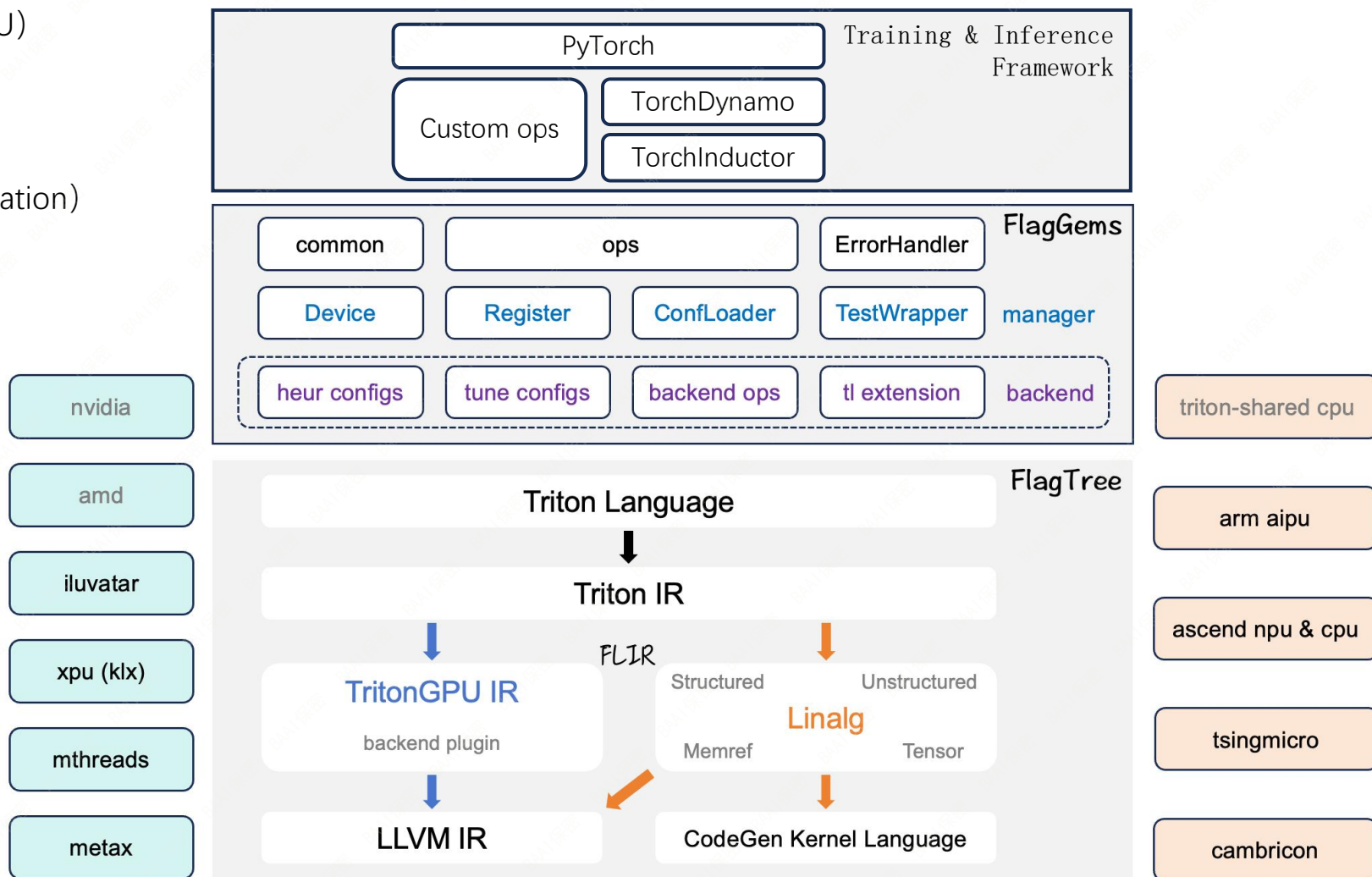
<https://github.com/FlagTree/flagtree>

<https://gitee.com/flagtree/flagtree>



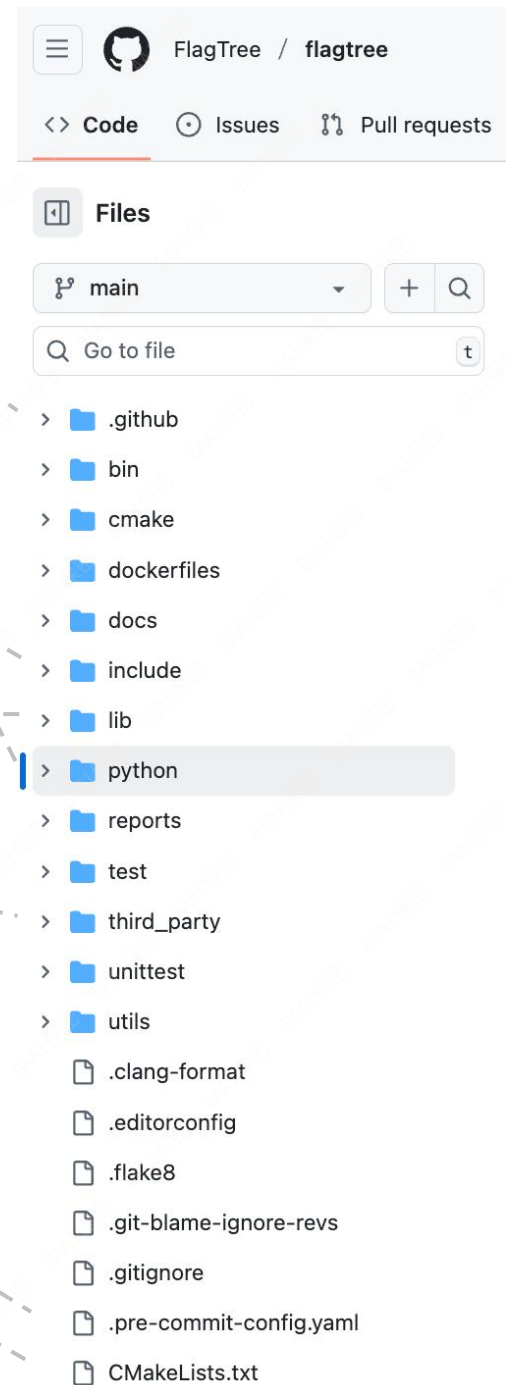
算子编译生态矩阵 (GPGPU & DSA/NPU)

1. 训推框架中的算子
2. 算子层 (Jit Function & Backend Integration)
3. 编程接口 (Triton Language)
4. 前端 (Front-End)
5. 中间层 (Mid-Layer)
6. 工程构建 (Building)



FlagTree 代码结构

- `.github`: 仓库管理配置, 包括 Issue 模板、PR 模板、CI / CD 配置等
- `python`: 编译时 (compiler)、运行时 (runtime) 的入口及调度代码, 后端管理代码 (backends), Triton 编程接口及前端, 构建入口 (setup.py)
- `include`: Dialect 的定义及其优化 (Transforms) 的声明, Dialect 间转换 (Conversion) 的声明
- `lib`: Dialect 的实现, 优化 (Transforms)、转换 (Conversion) 的实现
- `third_party`: 各芯片后端的接入 (backend) 及实现
- `.pre-commit-config.yaml`: 代码规范配置文件
- `CMakeLists.txt`: C++ 构建脚本配置文件
- `README.md`: 项目介绍文档



1. FlagTree 后端接入：工程构建

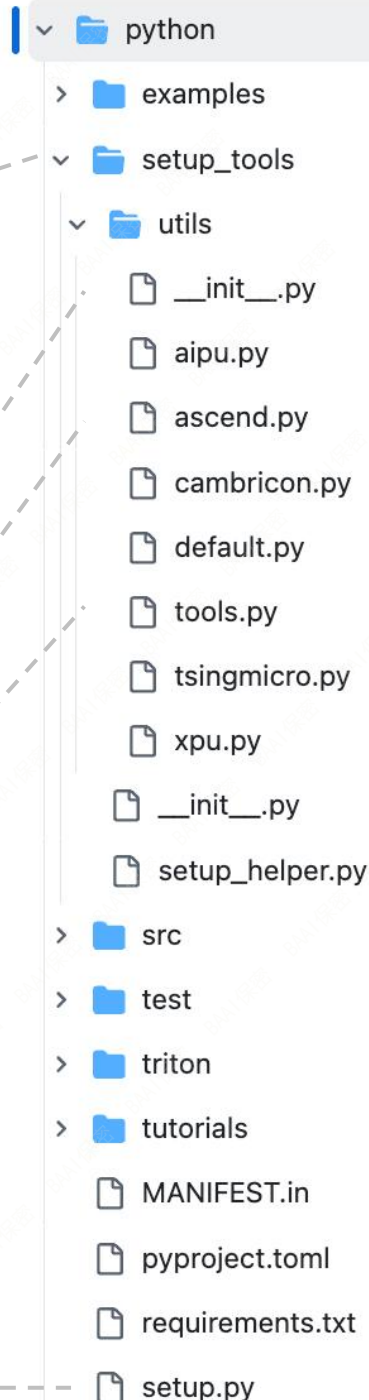
- `python/setup.py` 构建入口
 - 在 Triton 基础上添加后端构建功能
 - 多后端构建管理封装于 `setup_tools`
- `python/setup_tools/setup_helper.py`
 - 管理多后端构建
 - 包括多后端构建参数，三方库下载，安装打包范围，二进制插件、LLVM 等软件依赖下载及其缓存管理等
- `python/setup_tools/utils/__init__.py` 后端三方库具体下载地址及版本
- `python/setup_tools/utils/ascend.py` 构建包管理的后端差异化实现
- `python/setup_tools/utils/tools.py` 离线构建功能管理

```
from setuptools import Extension, setup
from setuptools.command.build_ext import build_ext
from setuptools.command.build_py import build_py
from dataclasses import dataclass

from distutils.command.install import install
from setuptools.command.develop import develop
from setuptools.command.egg_info import egg_info
from wheel.bdist_wheel import bdist_wheel

from setup_tools import setup_helper as helper
```

```
# arm aipu
cache.store(
    file="llvm-a66376b0-ubuntu-x64",
    condition=("aipu" == flagtree_backend),
    url="https://oaitriton.blob.core.windows.net/public/llvm-builds/llvm-a66376b0-ubuntu-x64.tar.gz",
    pre_hook=lambda: check_env('LLVM_SYSPATH'),
    post_hook=set_llvm_env,
```



1. FlagTree 后端接入：工程构建

- 构建命令：见项目仓库首页

```
apt install zlib1g zlib1g-dev libxml2 libxml2-dev # ubuntu  
cd python; python3 -m pip install -r requirements.txt
```

```
cd python  
export FLAGTREE_BACKEND=backendxxx  
python3 -m pip install . --no-build-isolation -v
```

iluvatar

```
# Recommended: Use Ubuntu 20.04  
mkdir -p ~/.flagtree/iluvatar; cd ~/.flagtree/iluvatar  
wget https://github.com/FlagTree/flagtree/releases/download/v0.3.0-build-deps/iluvatar-llvm18-  
tar zxvf iluvatar-llvm18-x86_64.tar.gz  
wget https://github.com/FlagTree/flagtree/releases/download/v0.3.0-build-deps/iluvatarTritonPl  
tar zxvf iluvatarTritonPlugin-cpython3.10-glibc2.30-glibcxx3.4.28-cxxabi1.3.12-ubuntu-x86_64.t  
cd ${YOUR_CODE_DIR}/flagtree/python  
export FLAGTREE_BACKEND=iluvatar  
python3 -m pip install . --no-build-isolation -v
```

2. FlagTree 后端接入: backends

- 后端发现机制 `backends/__init__.py`

```
def _discover_backends():
    backends = dict()
    root = os.path.dirname(__file__)
    for name in os.listdir(root):
        if not os.path.isdir(os.path.join(root, name)):
            continue
        if name.startswith('__'):
            continue
        compiler = _load_module(name, os.path.join(root, name, 'compiler.py'))
        driver = _load_module(name, os.path.join(root, name, 'driver.py'))
        backends[name] = Backend(_find_concrete_subclasses(compiler, BaseBackend),
                                _find_concrete_subclasses(driver, DriverBase))

    return backends
```

- 后端接入实现

- `third_party/nvidia/backend/compiler.py`
- `third_party/nvidia/backend/driver.py`

```
@dataclass(frozen=False)
class CUDAOptions:

class CUDABackend(BaseBackend):

class CudaLauncher(object):

class CudaDriver(GPUDriver):
```


2. FlagTree 后端接入: backends

- 后端发现机制 `backends/__init__.py`

```
def _discover_backends():
    backends = dict()
    root = os.path.dirname(__file__)
    for name in os.listdir(root):
        if not os.path.isdir(os.path.join(root, name)):
            continue
        if name.startswith('__'):
            continue
        compiler = _load_module(name, os.path.join(root, name, 'compiler.py'))
        driver = _load_module(name, os.path.join(root, name, 'driver.py'))
        backends[name] = Backend(_find_concrete_subclasses(compiler, BaseBackend),
                                _find_concrete_subclasses(driver, DriverBase))

    return backends
```

- 运行时获取活跃后端 `runtime/driver.py`

```
def _create_driver():
    actives = [x.driver for x in backends.values() if x.driver.is_active()]
    if len(actives) != 1:
        raise RuntimeError(f"{len(actives)} active drivers ({actives}). There should only be one.")
    return actives[0]()
```

`backend/driver.py`

3. FlagTree 后端接入: backend (driver)

- 后端接入实现 (驱动)

third_party/nvidia/backend/driver.py

```
class CudaLauncher(object):

    def __init__(self, src, metadata):
        ids = {"ids_of_const_exprs": src.fn.constexprs if hasattr(src, "fn") else tuple()}
        constants = src.constants if hasattr(src, "constants") else dict()
        cst_key = lambda i: src.fn.arg_names.index(i) if isinstance(i, str) else i
        constants = {cst_key(key): value for key, value in constants.items()}
        signature = {cst_key(key): value for key, value in src.signature.items()}
        src = make_launcher(constants, signature, ids)
        mod = compile_module_from_src(src, "__triton_launcher")
        self.launch = mod.launch

    def __call__(self, *args, **kwargs):
        self.launch(*args, **kwargs)
```

```
class CudaDriver(GPUDriver):
```

```
    def __init__(self):
        self.utils = CudaUtils() # TODO: make static
        self.launcher_cls = CudaLauncher ← compiler/compiler.py
        super().__init__()

    def get_current_target(self):
        device = self.get_current_device()
        capability = self.get_device_capability(device)
        capability = capability[0] * 10 + capability[1]
        warp_size = 32
        return GPUSubtarget("cuda", capability, warp_size)

    @staticmethod
    def is_active():
        import torch
        return torch.cuda.is_available() and (torch.version.hip is None)
```

runtime/driver.py ← --

3. FlagTree 后端接入: backend (driver)

- 后端接入实现 (驱动)

third_party/nvidia/backend/driver.py

```
class CudaLauncher(object):

    def __init__(self, src, metadata):
        ids = {"ids_of_const_exprs": src.fn.constexprs if hasattr(src, "fn") else tuple()}
        constants = src.constants if hasattr(src, "constants") else dict()
        cst_key = lambda i: src.fn.arg_names.index(i) if isinstance(i, str) else i
        constants = {cst_key(key): value for key, value in constants.items()}
        signature = {cst_key(key): value for key, value in src.signature.items()}
        src = make_launcher(constants, signature, ids)
        mod = compile_module_from_src(src, "__triton_launcher")
        self.launch = mod.launch

    def __call__(self, *args, **kwargs):
        self.launch(*args, **kwargs)
```

```
class CudaDriver(GPUDriver):
```

```
    def __init__(self):
        self.utils = CudaUtils() # TODO: make static
        self.launcher_cls = CudaLauncher ← compiler/compiler.py
        super().__init__()

    def get_current_target(self):
        device = self.get_current_device()
        capability = self.get_device_capability(device)
        capability = capability[0] * 10 + capability[1]
        warp_size = 32
        return GPUSubtarget("cuda", capability, warp_size)

    @staticmethod
    def is_active():
        import torch
        return torch.cuda.is_available() and (torch.version.hip is None)
```

runtime/driver.py ← --

3. FlagTree 后端接入: backend (launcher)

- 后端接入实现 (驱动 Launcher) [third_party/aipu/backend/driver.py](#)

[runtime/jit.py](#)

```
class JITFunction(KernelInterface[T]):
    def run(self, *args, grid, warmup, **kwargs):
        src = self.ASTSource(self, signature, constexprs, attrs)
        kernel = self.compile(src, target=target, options=options.__dict__)
        kernel.run(grid_0, grid_1, grid_2, stream, kernel.function, kernel.packed_metadata,
                    launch_metadata, self.CompiledKernel.launch_enter_hook, self.CompiledKernel.launch_exit_hook,
                    *bound_args.values())
        return kernel
```

[compiler/compiler.py](#)

```
def compile(src, target=None, options=None):
    options = backend.parse_options(dict(options or dict(), **extra_options))
    # create cache manager
    env_vars = get_cache_invalidating_env_vars()
    key = f"{triton_key()}-{src.hash()}-{backend.hash()}-{options.hash()}-{str}"
    hash = hashlib.sha256(key.encode("utf-8")).hexdigest()
    fn_cache_manager = get_cache_manager(hash)
    stages = dict()
    backend.add_stages(stages, options)
    if not isinstance(src, IRSource):
        context = ir.context()
        ir.load_dialects(context)
        backend.load_dialects(context)

    codegen_fns = backend.get_codegen_implementation(options)
    module_map = backend.get_module_map()
```

```
class CompiledKernel:
    def __init_handles(self):
        device = driver.active.get_current_device()
        # create launcher
        self.run = driver.active.launcher_cls(self.src, self.metadata) --> backend/driver.py

    def __getitem__(self, grid):
        self.__init_handles()

    def runner(*args, stream=None):
        if stream is None:
            device = driver.active.get_current_device()
            stream = driver.active.get_current_stream(device)
        launch_metadata = self.launch_metadata(grid, stream, *args)
        self.run(grid[0], grid[1], grid[2], stream, self.function, self.packed_metadata, launch_metadata,
                  CompiledKernel.launch_enter_hook, CompiledKernel.launch_exit_hook, *args)

    return runner
```


3. FlagTree 后端接入: backend (cuda launcher)

- 后端接入实现 (CudaLauncher) [third_party/nvidia/backend/driver.py](#)

```
class CudaLauncher(object):
```

```
    def __init__(self, src, metadata):
        ids = {"ids_of_const_exprs": src.fn.constexprs if hasattr(src, "fn") else tuple()}
        constants = src.constants if hasattr(src, "constants") else dict()
        cst_key = lambda i: src.fn.arg_names.index(i) if isinstance(i, str) else i
        constants = {cst_key(key): value for key, value in constants.items()}
        signature = {cst_key(key): value for key, value in src.signature.items()}
        src = make_launcher(constants, signature, ids)
        mod = compile_module_from_src(src, "__triton_launcher")
        self.launch = mod.launch
```

```
    def __call__(self, *args, **kwargs):
        self.launch(*args, **kwargs)
```

```
class CudaDriver(GPUDriver):
```

```
    def __init__(self):
        self.utils = CudaUtils() # TODO: make static
        self.launcher_cls = CudaLauncher ← -- compiler/compiler.py
        super().__init__()
```

```
    def get_current_target(self):
        device = self.get_current_device()
        capability = self.get_device_capability(device)
        capability = capability[0] * 10 + capability[1]
        warp_size = 32
        return GPUSubTarget("cuda", capability, warp_size)
```

```
    @staticmethod
    def is_active():
        import torch
        return torch.cuda.is_available() and (torch.version.hip is None)
```

[runtime/driver.py](#) ← --

3. FlagTree 后端接入: backend (aipu launcher)

- 后端接入实现 (AIPULauncher) [third_party/aipu/backend/driver.py](#)

➤ 低成本桥接 芯片原有算子编译栈 (codegen)

```
class AIPULauncher(object):  
    def __call__(self, gridx_size, gridy_size, gridz_size, stream, function, *args):  
        ex = pickle.loads(function)  
        ex(*(np_args + tail_args))
```

```
class AIPUDriver(DriverBase):
```

```
    def __init__(self):  
        self.utils = AIPUUtils() # TODO: make static  
        self.launcher_cls = AIPULauncher
```

← - - - - - compiler/compiler.py

runtime/driver.py ← - -

```
@staticmethod  
def is_active():  
    import torch  
    from torch.utils import cpp_extension
```

```
try:  
    torch.aipu.is_available()  
except AttributeError:  
    current_dir = Path(__file__).resolve().parent  
    extra_ldflags = [f"-L{x.strip()}" for x in os.getenv("LD_LIBRARY_PATH", "").split(":") if x.strip() != ""]  
    extra_ldflags.append("-laipudrv")  
    module = cpp_extension.load(  
        name="aipu", sources=[current_dir / "aipu_torch_dev.cpp"],  
        extra_include_paths=[os.getenv("ZHOUYI_LINUX_DRIVER_HOME") + "/driver/umd/include"],  
        extra_ldflags=extra_ldflags, verbose=True)  
  
    torch.utils.rename_privateuse1_backend("aipu")  
    torch._register_device_module("aipu", module)  
    torch.utils.generate_methods_for_privateuse1_backend(for_storage=True)  
    return torch.aipu.is_available()
```

对 pytorch 的后端扩展

```
class AIPUBackend(BaseBackend):  
    def make_aipubin(mod, metadata, opt):  
        ex = codegenAIPU(mod)  
        metadata["name"] = ex._func_name  
        metadata["shared"] = 1  
        return pickle.dumps(ex)
```

[third_party/aipu/backend/compiler.py](#)

4. FlagTree 后端接入: backend (compiler)

- 后端接入实现 (JITFunction 入口)

runtime/jit.py

```
class JITFunction(KernelInterface[T]):
    def run(self, *args, grid, warmup, **kwargs):
        src = self.ASTSource(self, signature, constexprs, attrs)
        kernel = self.compile(src, target=target, options=options.__dict__)
        kernel.run(grid_0, grid_1, grid_2, stream, kernel.function, kernel.packed_metadata,
                    launch_metadata, self.CompiledKernel.launch_enter_hook, self.CompiledKernel.launch_exit_hook,
                    *bound_args.values())
        return kernel
```

compiler/compiler.py

```
def compile(src, target=None, options=None):
    options = backend.parse_options(dict(options or dict(), **extra_options))
    # create cache manager
    env_vars = get_cache_invalidating_env_vars()
    key = f"{triton_key()}-{src.hash()}-{backend.hash()}-{options.hash()}-{str}"
    hash = hashlib.sha256(key.encode("utf-8")).hexdigest()
    fn_cache_manager = get_cache_manager(hash)
    stages = dict()
    backend.add_stages(stages, options)
    if not isinstance(src, IRSource):
        context = ir.context()
        ir.load_dialects(context)
        backend.load_dialects(context)

    codegen_fns = backend.get_codegen_implementation(options)
    module_map = backend.get_module_map()
```

```
class CompiledKernel:
    def __init__(self):
        device = driver.active.get_current_device()
        # create launcher
        self.run = driver.active.launcher_cls(self.src, self.metadata) --> backend/driver.py

    def __getitem__(self, grid):
        self.__init_handles()

    def runner(*args, stream=None):
        if stream is None:
            device = driver.active.get_current_device()
            stream = driver.active.get_current_stream(device)
        launch_metadata = self.launch_metadata(grid, stream, *args)
        self.run(grid[0], grid[1], grid[2], stream, self.function, self.packed_metadata, launch_metadata,
                  CompiledKernel.launch_enter_hook, CompiledKernel.launch_exit_hook, *args)

    return runner
```


4. FlagTree 后端接入: backend (compiler)

- 后端接入实现 (编译时) `third_party/backendxxx/backend/compiler.py`

```
from triton._C.libtriton import backendxxx
```

```
class CUDABackend (BaseBackend):
```

```
    def parse_options(self, opts) -> Any:
```

```
        return CUDAOptions(**args)
```

```
    def load_dialects(self, ctx):
```

```
        backendxxx.load_dialects(ctx) triton_backendxxx.py
```

```
    def make_ttir(mod, metadata, opt): ...
```

```
    def make_ttgir(mod, metadata, opt, capability): ...
```

```
    def make_llir(src, metadata, options, capability): ...
```

```
    def make_cubin(src, metadata, options, capability):
```

```
    def add_stages(self, stages, options):
```

```
        stages["ttir"] = lambda src, metadata: self.make_ttir(src, metadata, options)
```

```
        stages["ttgir"] = lambda src, metadata: self.make_ttgir(src, metadata, options, self.capability)
```

```
        stages["llir"] = lambda src, metadata: self.make_llir(src, metadata, options, self.capability)
```

```
        stages["cubin"] = lambda src, metadata: self.make_cubin(src, metadata, options, self.capability)
```

```
@dataclass(frozen=False)
```

```
class CUDAOptions:
```

```
    num_warps: int = 4
```

```
    num_ctas: int = 1
```

```
    num_stages: int = 3
```

```
    # maxnreg corresponds to the ptx parameter .maxnreg, which controls the  
    # maximum number of 32-bit registers used by one thread.
```

```
    maxnreg: Optional[int] = None
```

```
    cluster_dims: tuple = (1, 1, 1)
```

```
    enable_fp_fusion: bool = True
```

```
    allow_fp8e4nv: bool = False
```

```
    allow_fp8e4b15: bool = False
```

```
@staticmethod
```

```
def make_ttir(mod, metadata, opt):
```

```
    pm = ir.pass_manager(mod.context)
```

```
    pm.enable_debug()
```

```
    passes.common.add_inliner(pm)
```

```
    passes.ttir.add_rewrite_tensor_pointer(pm)
```

```
    passes.ttir.add_combine(pm)
```

```
    passes.common.add_canonicalizer(pm)
```

```
    passes.ttir.add_reorder_broadcast(pm)
```

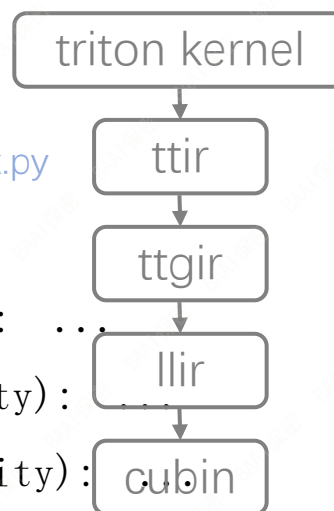
```
    passes.common.add_cse(pm)
```

```
    passes.common.add_lcm(pm)
```

```
    passes.common.add_symbol_dce(pm)
```

```
    pm.run(mod)
```

```
    return mod
```



5. FlagTree 后端接入: GPGPU

- nvidia 提供了一套成熟的编译路径范例, 厂商按需在各阶段打补丁或实现 FlagTree 的统一抽象接口即可
 - Triton JIT Function
 - Triton IR (ttir)
 - TritonGPU IR (ttgir)
 - LLVM IR (llir)

```
def DotOperandEncodingAttr : DistributedEncoding<"DotOperandEncoding", "dot_operand_encoding"> {  
  let mnemonic = "dot_op";  
  
  let parameters = (  
    ins  
    "unsigned":$opIdx,  
    "Attribute":$parent,  
    DefaultValuedParameter<"unsigned", "0">:$kWidth,  
    "unsigned":$useSme  
  );  
};
```

5. FlagTree 后端接入: GPGPU

- 实现 FlagTree 的统一抽象接口
 - `runtime/driver.py` 提供了 `flagtree_backend_specialization` 接口, 可在差异化实现之处调用

```
python/triton/runtime/driver.py
```

```
+ # flagtree backend specialization
+ def flagtree_backend_specialization(function_name: str, *args, **kwargs):
+     flagtree_backend_specialization = driver.active.flagtree_backend_specialization
+     if hasattr(flagtree_backend_specialization, function_name):
+         func = getattr(flagtree_backend_specialization, function_name)
+         return func(*args, **kwargs)
+     return None
```

```
python/triton/compiler/code_generator.py
```

```
def kernel_suffix(signature, specialization):
    # suffix format:
    # <argid><'c' if equal to 1><'d' if divisible by 16><'e' if divisible by 8>
    suffix = ''
    for i, _ in enumerate(signature):
        suffix += str(i)
        if i in specialization.equal_to_1:
            suffix += 'c'
        if i in specialization.divisible_by_16:
            suffix += 'd'
```

```
+ # flagtree backend specialization
+ from triton.runtime.driver import flagtree_backend_specialization
+ suffix = flagtree_backend_specialization("kernel_suffix_by_divisibility", specialization, i, suffix) or suffix
return suffix
```

5. FlagTree 后端接入: GPGPU

- 实现 FlagTree 的统一抽象接口
 - `backend/flagtree_backend_specialization` 进行差异化实现, 注册于 `backend/driver.py` 即可

```
..._party/iluvatar/backend/flagtree_backend_specialization/triton/compiler/code_generator.py
```

```
@@ -0,0 +1,13 @@  
+ def kernel_suffix_by_divisibility(specialization, i, suffix: str) -> str:  
+     if i in specialization.divisible_by_8:  
+         suffix += 'e'  
+     return suffix
```

```
third_party/iluvatar/backend/driver.py
```

```
class CudaDriver(GPUDriver):  
  
    def __init__(self):  
        self.utils = CudaUtils() # TODO: make static  
        self.launcher_cls = CudaLauncher  
+     from triton.backends.iluvatar import flagtree_backend_specialization  
+     self.flagtree_backend_specialization = flagtree_backend_specialization  
        super().__init__()
```

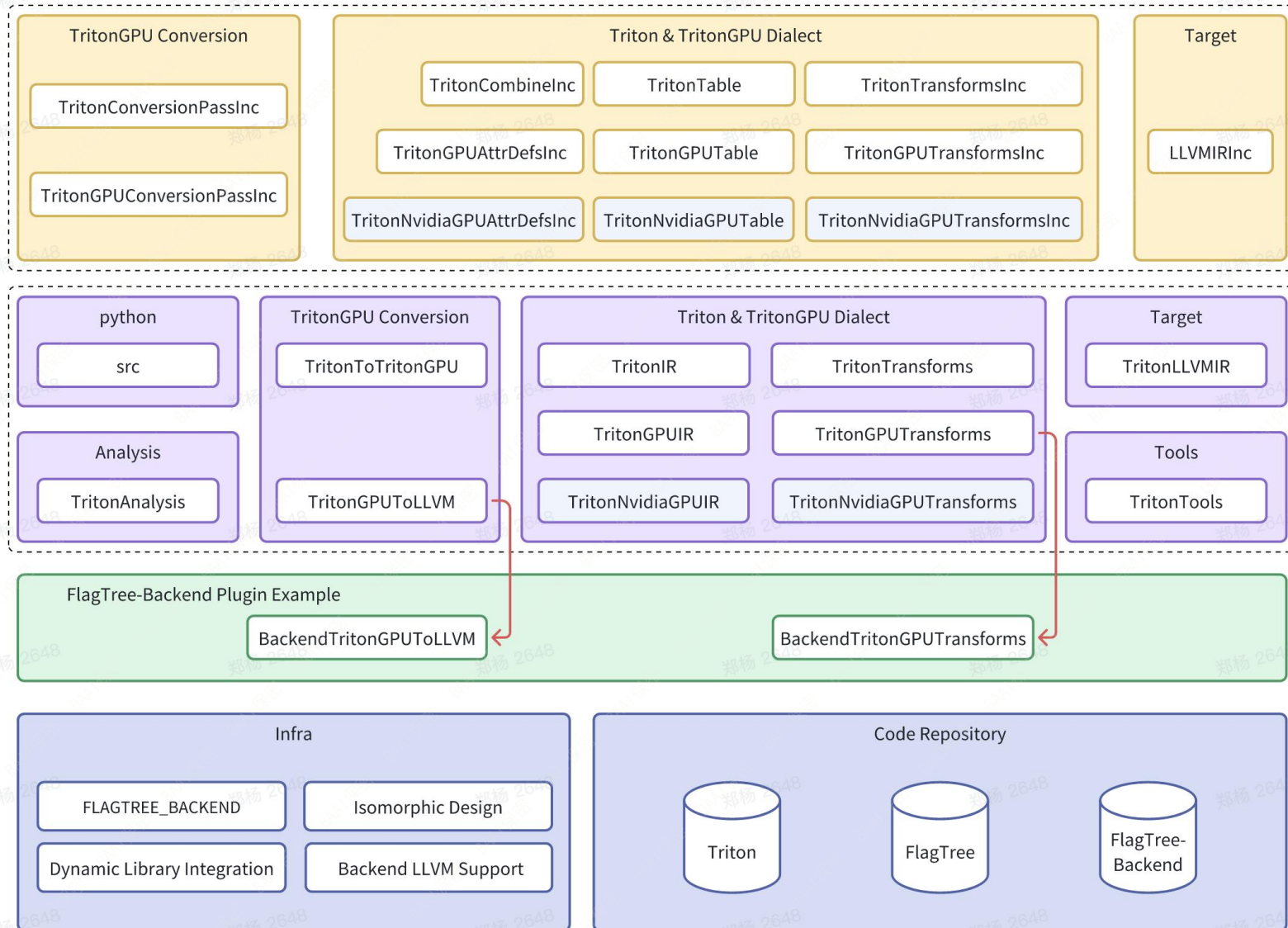

5. FlagTree 后端接入: GPGPU Plugin

- 芯片高差异度模块的插件化
- 主仓库使用动态库/静态库接入
- 非公开仓库负责生成二进制库, 代码结构仍可与主仓库保持同构设计, 代码仅用宏区分, 构建命令使用环境变量区分

[iluvatar](#)

```
# Recommended: Use Ubuntu 20.04
mkdir -p ~/.flagtree/iluvatar; cd ~/.flagtree/iluvatar
wget https://github.com/FlagTree/flagtree/releases/download,
tar zxvf iluvatar-llvm18-x86_64.tar.gz
wget https://github.com/FlagTree/flagtree/releases/download,
tar zxvf iluvatarTritonPlugin-cpython3.10-glibc2.30-glibcxx,
cd ${YOUR_CODE_DIR}/flagtree/python
export FLAGTREE_BACKEND=iluvatar
python3 -m pip install . --no-build-isolation -v
```

`export FLAGTREE_PLUGIN=iluvatar`



6. FlagTree 后端接入: DSA / NPU

- 编译阶段 **GPGPU** (TritonGPU) vs **DSA / NPU** (Linalg)

[third_party/nvidia/backend/compiler.py](#)

```
def add_stages(self, stages, options):
    capability = self._parse_arch(options.arch)
    stages["ttir"] = lambda src, metadata: self.make_ttir(src, metadata, options)
    stages["ttgir"] = lambda src, metadata: self.make_ttgir(src, metadata, options, capability)
    stages["llir"] = lambda src, metadata: self.make_llir(src, metadata, options, capability)
    stages["ptx"] = lambda src, metadata: self.make_ptx(src, metadata, options, self.target.arch)
    stages["cubin"] = lambda src, metadata: self.make_cubin(src, metadata, options, self.target.arch)
```

[third_party/aipu/backend/compiler.py](#)

```
def add_stages(self, stages, options):
    stages["ttir"] = lambda src, metadata: self.make_ttir(src, metadata, options)
    stages["linalg"] = lambda src, metadata: self.make_linalg(src, metadata, options)
    stages["bin"] = lambda src, metadata: self.make_aipubin(src, metadata, options)
```

6. FlagTree 后端接入: DSA / NPU

- 编译阶段
 - Triton JIT Function

[python/tutorials/01-vector-add.py](#)

```
1 @triton.jit
2 def add_kernel(x_ptr, y_ptr, output_ptr, n_elements, BLOCK_SIZE=1024: tl.constexpr):
3     pid = tl.program_id(axis=0)
4     block_start = pid * BLOCK_SIZE
5     offsets = block_start + tl.arange(0, BLOCK_SIZE)
6     mask = offsets < n_elements
7     x = tl.load(x_ptr + offsets, mask=mask)
8     y = tl.load(y_ptr + offsets, mask=mask)
9     output = x + y
10    tl.store(output_ptr + offsets, output, mask=mask)
```



6. FlagTree 后端接入: DSA / NPU

- 编译阶段

- make_ttir (tt + arith)

- `triton.jit => tt.func`
 - 参数 `tl.tensor => tt.ptr`
 - 去除常量参数
- `tl.constexpr => arith.constant`
- `tl.program_id => tt.get_program_id`
- `tl.arange => tt.make_range : tensor`
- `tl.load`、`tl.store` => `tt.load`、`tt.store`
- 数值的标量或张量计算 => `arith.muli : i32`、`arith.addi : tensor<i32>`、`arith.addf : tensor<f32>`
- 数值的标量与张量计算前需展开 => `tt.splat : i32 -> tensor`
- 指针的标量与张量计算前需展开 => `tt.splat : tt.ptr -> tensor`
- 指针地址相加 `x_ptr + offsets` => `tt.addptr : tensor<tt.ptr>, tensor<i32>`

6. FlagTree 后端接入: DSA / NPU

- 编译阶段 DSA / NPU

- make_linalg (linalg + arith + memref + bufferization + tensor)

- `tt.func => func.func` (20. TritonArithToLinalg)
 - 参数 `tt.ptr` => `memref` (23-25. TritonPtrToMemref)
 - 添加 6 个参数 `num_programs(0/1/2)`、`program_id(0/1/2)` <= `tt.get_program_id` (20. TritonArithToLinalg)
- 标量用于下标偏移量 => `arith.index_cast : i32 to index` (12-16. TritonToStructured)
- `tt.load in_ptr + (start,end), mask` 将数据从 memref 通过 subview + copy + bufferization 加载到 tensor =>
 - 分配输出 `%alloc = memref.alloc() : memref`
 - `tt.addptr in_ptr, (start,end) =>`
`%cast_in_ptr = memref.reinterpret_cast in_ptr to offset:[start], sizes:[end-start], strides:[1]`
`: memref to memref`
 - `tt.make_range {end,start} : tensor`、`mask = ... =>`
`%subview_src = memref.subview %cast_in_ptr[0] [(end-start) & mask] [1] : memref to memref`
`%subview_dst = memref.subview %alloc[0] [(end-start) & mask] [1] : memref to memref`
 - `tt.load =>`
`memref.copy %subview_src, %subview_dst : memref to memref`
`in_value = bufferization.to_tensor %alloc restrict writable : memref to tensor`
 - 在 12-16. TritonToStructured 阶段, `tt.make_range`、`tt.load` 转换成了 `"tts.load"(tts.make_tptr, end-start)`
`<{operandSegmentSizes, static_mask_dims}> : (tensor<tt.ptr<f32>>, index) -> tensor<f32>`
 - 在 21. StructuredToMemref 阶段, `"tts.load"` 转换成最终形态。

6. FlagTree 后端接入: DSA / NPU

- 编译阶段 DSA / NPU

- make_linalg (linalg + arith + memref + bufferization + tensor)

- tt.store out_ptr + (start,end), out_value, mask 将数据从 tensor 通过 subview + extract_slice 保存至 memref =>
 - tt.addptr out_ptr, (start,end) =>
`%cast_out_ptr = memref.reinterpret_cast out_ptr to offset:[start], sizes:[end-start], strides:[1] : memref to memref`
 - tt.make_range {end,start} : tensor、mask = ... =>
`%subview_out = memref.subview %cast_out_ptr[0] [(end-start) & mask] [1] : memref to memref`
 - tt.store =>
`%extracted_slice = tensor.extract_slice out_value[0] [(end-start) & mask] [1] : tensor to tensor`
`bufferization.materialize_in_destination %extracted_slice in writable %subview_out`
 - 在 12-16. TritonToStructured 阶段, tt.make_range、tt.store 转换成了 "tts.store"(tts.make_tptr, value, end-start) <{static_mask_dims}> : (tensor<tt.ptr<f32>>, tensor<f32>, index) -> ()
 - 在 21. StructuredToMemref 阶段, "tts.store" 转换成最终形态。
- 结构化的张量逐点运算 arith.addf x, y : tensor => (20. createTritonArithToLinalgPass)
`#map = affine_map<(d0) -> (d0)> 两个输入、一个输出使用相同的索引映射, 表示元素级别的逐点映射`
`linalg.generic {indexing_maps=[#map,#map,#map], iterator_types=["parallel"]} ins(x, y) outs(x) {`
`^bb0(%in_x: f32, %in_y: f32, %out: f32): 对应三个张量的单个元素`
`%scalar_val = arith.addf %in_x, %in_y : f32 标量加法`
`linalg.yield %scalar_val : f32`
`}` -> tensor

7. FlagTree 后端接入: dialect & pass 注册

- 后端 dialect & pass 注册 init_triton_backendxxx

python/src/main.cc

```
#define DECLARE_BACKEND(name) void init_triton_##name(pybind11::module &m);

#define INIT_BACKEND(name) init_triton_##name(m.def_submodule(#name));

void init_triton_env_vars(pybind11::module &m);
void init_triton_ir(pybind11::module &m);
void init_triton_llvm(pybind11::module &m);
void init_triton_interpreter(pybind11::module &m);
void init_triton_passes(pybind11::module &m);
void init_triton_stacktrace_hook(pybind11::module &m);
FOR_EACH_P(DECLARE_BACKEND, TRITON_BACKENDS_TUPLE)

PYBIND11_MODULE(libtriton, m) {
    m.doc() = "Python bindings to the C++ Triton API";
    init_triton_stacktrace_hook(m);
    init_triton_env_vars(m);
    init_triton_ir(m.def_submodule("ir"));
    init_triton_passes(m.def_submodule("passes"));
    init_triton_interpreter(m.def_submodule("interpreter"));
    init_triton_llvm(m.def_submodule("llvm"));
    FOR_EACH_P(INIT_BACKEND, TRITON_BACKENDS_TUPLE)
}
```

backend/compiler.py ← --

```
void init_triton_aipu_passes_convert(py::module &m) {
    ADD_PASS_WRAPPER_0("add_linalg_to_std", createConvertLinalgToStandardPass);
    ADD_PASS_WRAPPER_0("add_one_shot_bufferize",
        bufferization::createOneShotBufferizePass);
    ADD_PASS_WRAPPER_0("add_buff_to_memref",
        createConvertBufferizationToMemRefPass);
    ADD_PASS_WRAPPER_0("add_triton_to_linalg", triton::createTritonToLinalgPass);
    ADD_PASS_WRAPPER_0("add_affine_to_std", createLowerAffinePass);
    ADD_PASS_WRAPPER_0("add_triton_to_linalg_pipeline",
        triton::createTritonToLinalgExperimentalPass);
}

void init_triton_aipu(py::module &m) {
    init_triton_aipu_common(m.def_submodule("common"));
    auto passes = m.def_submodule("passes");
    init_triton_aipu_passes_convert(passes.def_submodule("convert"));
    // load dialects
    m.def("load_dialects", [](mlir::MLIRContext &context) {
        using namespace mlir;
        DialectRegistry registry;
        registry.insert<arith::ArithDialect, linalg::LinalgDialect,
            tensor::TensorDialect, vector::VectorDialect,
            func::FuncDialect>();

        arith::registerBufferizableOpInterfaceExternalModels(registry);
        linalg::registerAllDialectInterfaceImplementations(registry);
        tensor::registerBufferizableOpInterfaceExternalModels(registry);
        bufferization::func_ext::registerBufferizableOpInterfaceExternalModels(
            registry);
        func::registerAllExtensions(registry);
        scf::registerBufferizableOpInterfaceExternalModels(registry);
        context.appendDialectRegistry(registry);
        context.loadAllAvailableDialects();
    });
    // register passes here
}
```

3. FlagTree 后端接入: pytorch 后端扩展

- 后端接入实现 (AIPULauncher) [third_party/aipu/backend/driver.py](#)

➤ 低成本桥接 芯片原有算子编译栈 (codegen)

```
class AIPULauncher(object):  
    def __call__(self, gridx_size, gridy_size, gridz_size, stream, function, *args):  
        ex = pickle.loads(function)  
        ex(*(np_args + tail_args))
```

```
class AIPUDriver(DriverBase):
```

```
    def __init__(self):  
        self.utils = AIPUUtils() # TODO: make static  
        self.launcher_cls = AIPULauncher
```

← - - - - - compiler/compiler.py

runtime/driver.py ← - -

```
@staticmethod  
    def is_active():  
        import torch  
        from torch.utils import cpp_extension
```

```
    try:  
        torch.aipu.is_available()  
    except AttributeError:  
        current_dir = Path(__file__).resolve().parent  
        extra_ldflags = [f"-L{x.strip()}" for x in os.getenv("LD_LIBRARY_PATH", "").split(":") if x.strip() != ""]  
        extra_ldflags.append("-laipudrv")  
        module = cpp_extension.load(  
            name="aipu", sources=[current_dir / "aipu_torch_dev.cpp"],  
            extra_include_paths=[os.getenv("ZHOUYI_LINUX_DRIVER_HOME") + "/driver/umd/include"],  
            extra_ldflags=extra_ldflags, verbose=True)  
  
        torch.utils.rename_privateuse1_backend("aipu")  
        torch._register_device_module("aipu", module)  
        torch.utils.generate_methods_for_privateuse1_backend(for_storage=True)  
    return torch.aipu.is_available()
```

对 pytorch 的后端扩展

```
class AIPUBackend(BaseBackend):  
    def make_aipubin(mod, metadata, opt):  
        ex = codegenAIPU(mod)  
        metadata["name"] = ex._func_name  
        metadata["shared"] = 1  
        return pickle.dumps(ex)
```

[third_party/aipu/backend/compiler.py](#)

8. FlagTree 后端接入: pytorch 后端扩展

- pytorch 后端标准扩展范例 `third_party/aipu/backend/aipu_torch_dev.cpp`

```
TORCH_LIBRARY_IMPL(aten, PrivateUse1, m) {
  m.impl("empty.memory_format", &custom_empty_symint);
  m.impl("empty_strided", &custom_empty_strided);
  m.impl("as_strided", native::as_strided_tensorimpl);
  m.impl("aten::view", &aipu_view);
  m.impl("aten::uniform_", &random_kernel<uniform_real_distribution>);
  m.impl("aten::normal_", &random_kernel<normal_distribution>);
  m.impl("aten::_copy_from", &aipu_copy_from);
  m.impl("aten::_copy_from_and_resize", &aipu_copy_from_and_resize);
  m.impl("aten::random_.from",
        &random_from_to_kernel<uniform_int_from_to_distribution>);
  m.impl("aten::_local_scalar_dense", &_local_scalar_dense_aipu);
  m.impl("aten::fill_.Scalar", &fill_scalar_aipu);
}
```

```
// Register the autograd dispatch key for operators that have no dispatches
TORCH_LIBRARY_IMPL(aten, AutogradPrivateUse1, m) {
  m.impl("isfinite", torch::autograd::autogradNotImplementedFallback());
}
```

```
PYBIND11_MODULE(TORCH_EXTENSION_NAME, m) {
  m.def("device_count", &aipu::device_count, "aipu device count");
  m.def("is_available", &aipu::is_available, "aipu is available");
  m.def("current_device", &aipu::current_device, "aipu current device");
  m.def("_is_in_bad_fork", []() { return py::bool_(false); });
  m.def("manual_seed_all", [](int seed) { std::srand(seed); });
  m.attr("default_generators") = &default_generators;

  py::class_<DeviceGuard>(m, "_DeviceGuard", py::module_local())
    .def(py::init(
      [](int index) { return std::make_unique<DeviceGuard>(index, -1); }))
    .def("__enter__", [ ](_DeviceGuard &self) { ; })
    .def("__exit__",
      [ ](_DeviceGuard &self, pybind11::object type, pybind11::object value,
        pybind11::object traceback) { return py::bool_(false); });

  py::class_<Device>(m, "device", py::module_local())
    .def(py::init(
      [ ](c10::Device device) { return std::make_unique<Device>(device); }))
    .def(py::init([ ](int index) { return std::make_unique<Device>(index); }))
    .def("__enter__", [ ](_Device &self) { ; })
    .def("__exit__",
      [ ](_Device &self, pybind11::object type, pybind11::object value,
        pybind11::object traceback) { return py::bool_(false); });
}
```

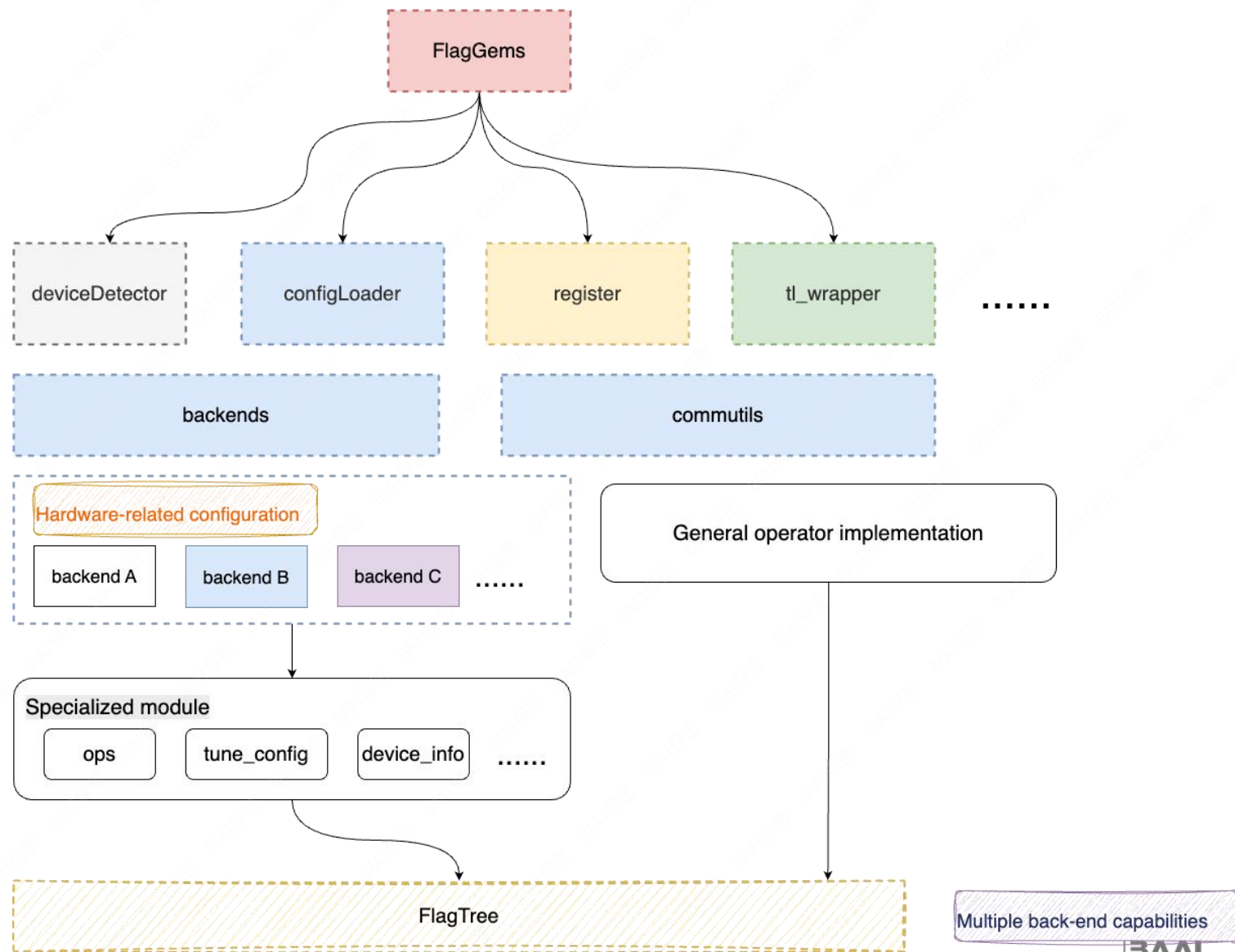

9. FlagGems 后端接入：算子库与编译器共建

- 算子库多后端管理

- PyTorch 功能函数接口、
TritonLanguage 原语接口等标准化
- ATen 算子拦截注册、私有注册键值
- 运行设备保护、张量设备指定
- 错误信息管理、异常处理
- 导入模块时自动进行后端检测

- 定制自动调参空间 & 启发式调参 & 算子实现

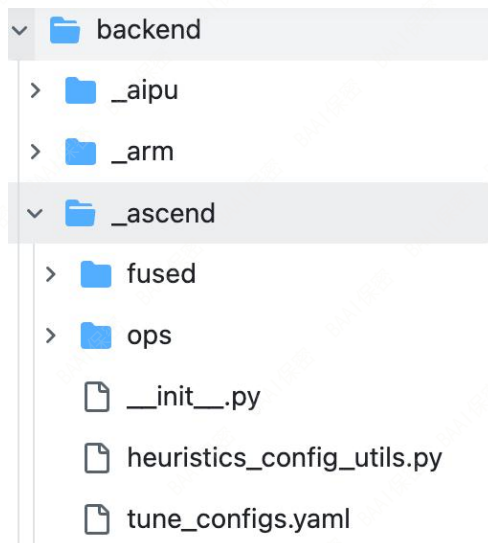
- 支持后端根据硬件特性调整配置参数与算子代码
- 支持后端禁用性能低洼算子、不予注册
- 支持用户手动选择禁用算子



9. FlagGems 后端接入：算子库与编译器共建

- 定制自动调参空间 & 启发式调参 & 算子实现

- 支持后端根据硬件特性调整配置参数与算子代码
- 支持后端禁用性能低洼算子、不予注册
- 支持用户手动选择禁用算子



src/flag_gems/runtime/backend/_ascend/heuristics_config_utils.py

```
def argmin_heur_block_n(args):  
    return min(4096, triton.next_power_of_2(args["N"]))
```

```
def bmm_heur_divisible_m(args):  
    return args["M"] % args["TILE_M"] == 0
```

src/flag_gems/runtime/backend/_ascend/tune_configs.yaml

```
gather:  
- gen: true  
  param_map:  
    META:  
      BLOCK_M: block_m  
      BLOCK_N: block_n  
      num_warps: 4  
    block_m:  
      - 1  
      - 2  
      - 4  
      - 8  
    block_n:  
      - 256  
      - 512  
      - 1024  
      - 2048
```



Thanks!

邮件: job@baai.ac.cn

电话: 010 - 6893 3383

地址: 北京市海淀区成府路150号智源大厦



智源公众号